Architecture 1001: x86-64 Assembly ShiftExample1.c

Xeno Kovah – 2021
xeno@darkmentor.com

All materials is licensed under a Creative Commons “Share Alike” license.

- http://creativecommons.org/licenses/by-sa/4.0/

You are free:

to Share — to copy, distribute and transmit the work

to Remix — to adapt the work

Under the following conditions:

Attribution — You must attribute the work in the manner specified by the author or licensor (but not in any way that suggests that they endorse you or your use of the work).

Share Alike — If you alter, transform, or build upon this work, you may distribute the resulting work only under the same, similar or a compatible license.

Attribution condition: You must indicate that derivative work
"Is derived from Xeno Kovah's 'Architecture 1001: x86-64 Assembly’ class, available at https://ost.fyi"

ShiftExample1.c

```
void main()
{
   unsigned int a, b, c;
   a = 0x5;
   b = a << 4;
   c = b >> 3;
   return c;
}
```

```
main:
0000000140001000  sub rsp,18h
0000000140001004  mov dword ptr [rsp],5
000000014000100B  mov eax,dword ptr [rsp]
000000014000100E  shl eax,4
0000000140001011  mov dword ptr [rsp+4],eax
0000000140001015  mov eax,dword ptr [rsp+4]
0000000140001019  shr eax,3
000000014000101C  mov dword ptr [rsp+8],eax
0000000140001020  mov eax,dword ptr [rsp+8]
0000000140001024  add rsp,18h
0000000140001028  ret
```

ShiftExample1.c

```c
void main()
{
   unsigned int a, b, c;
   a = 0x5;
   b = a << 4;
   c = b >> 3;
   return c;
}
```

```
main:
0000000140001000  sub rsp,18h
0000000140001004  mov dword ptr [rsp],5
000000014000100B  mov eax,dword ptr [rsp]
000000014000100E  shl eax,4
0000000140001011  mov dword ptr [rsp+4],eax
0000000140001015  mov eax,dword ptr [rsp+4]
0000000140001019  shr eax,3
000000014000101C  mov dword ptr [rsp+8],eax
0000000140001020  mov eax,dword ptr [rsp+8]
0000000140001024  add rsp,18h
0000000140001028  ret
```

SHL - Shift Logical Left

- Can be explicitly used with the C “<<” operator
- First operand (source and destination) is an r/mX
- Second operand is either cl (lowest byte of rcx), or a 1 byte immediate. The 2nd operand is the number of places to shift
- It **multiplies** the register by 2 for each place the value is shifted. More efficient than a multiply instruction
- Bits shifted off the left hand side are “shifted into” (set) the carry flag (CF)
- For purposes of determining if the CF is set at the end, think of it as n independent 1 bit shifts

shl bl, 2

	00110011b (bl - 0x33)
result	11001100b (bl - 0xCC) CF = 0

shl bl, 3

	00110011b (bl - 0x33)
result	10011000b (bl - 0x98) CF = 1

SHR - Shift Logical Right

- Can be explicitly used with the C “>>” operator
- First operand (source and destination) is an r/mX
- Second operand is either cl (lowest byte of rcx), or a 1 byte immediate. The 2nd operand is the number of places to shift
- It **divides** the register by 2 for each place the value is shifted. More efficient than a divide instruction
- Bits shifted off the right hand side are “shifted into” (set) the carry flag (CF)
- For purposes of determining if the CF is set at the end, think of it as n independent 1 bit shifts

shr bl, 2

	00110011b (bl - 0x33)
result	00001100b (bl - 0x0C) CF = 1

shr bl, 3

	00110011b (bl - 0x33)
result	00000110b (bl - 0x06) CF = 0



STOP

&&

STEP THROUGH

THE ASSEMBLY

&&

CHECK YOUR UNDERSTANDING



STOP

&&

STEP THROUGH

THE ASSEMBLY

&&

CHECK YOUR UNDERSTANDING

ShiftExample1.c Takeaways:

- Shifts in C are shifts in asm :P

```
void main()
{
  unsigned int a, b, c;
  a = 0x5;
  b = a << 4;
  c = b >> 3;
  return c;
}
```

```
main:
0000000140001000  sub rsp,18h
0000000140001004  mov dword ptr [rsp],5
000000014000100B  mov eax,dword ptr [rsp]
000000014000100E  shl eax,4
0000000140001011  mov dword ptr [rsp+4],eax
0000000140001015  mov eax,dword ptr [rsp+4]
0000000140001019  shr eax,3
000000014000101C  mov dword ptr [rsp+8],eax
0000000140001020  mov eax,dword ptr [rsp+8]
0000000140001024  add rsp,18h
0000000140001028  ret
```

Instructions we now know (24)

- NOP
- PUSH/POP
- CALL/RET
- MOV
- ADD/SUB
- IMUL
- MOVZX/MOVSX
- LEA
- JMP/Jcc (family)
- CMP/TEST
- AND/OR/XOR/NOT
- INC/DEC
- **SHR/SHL**

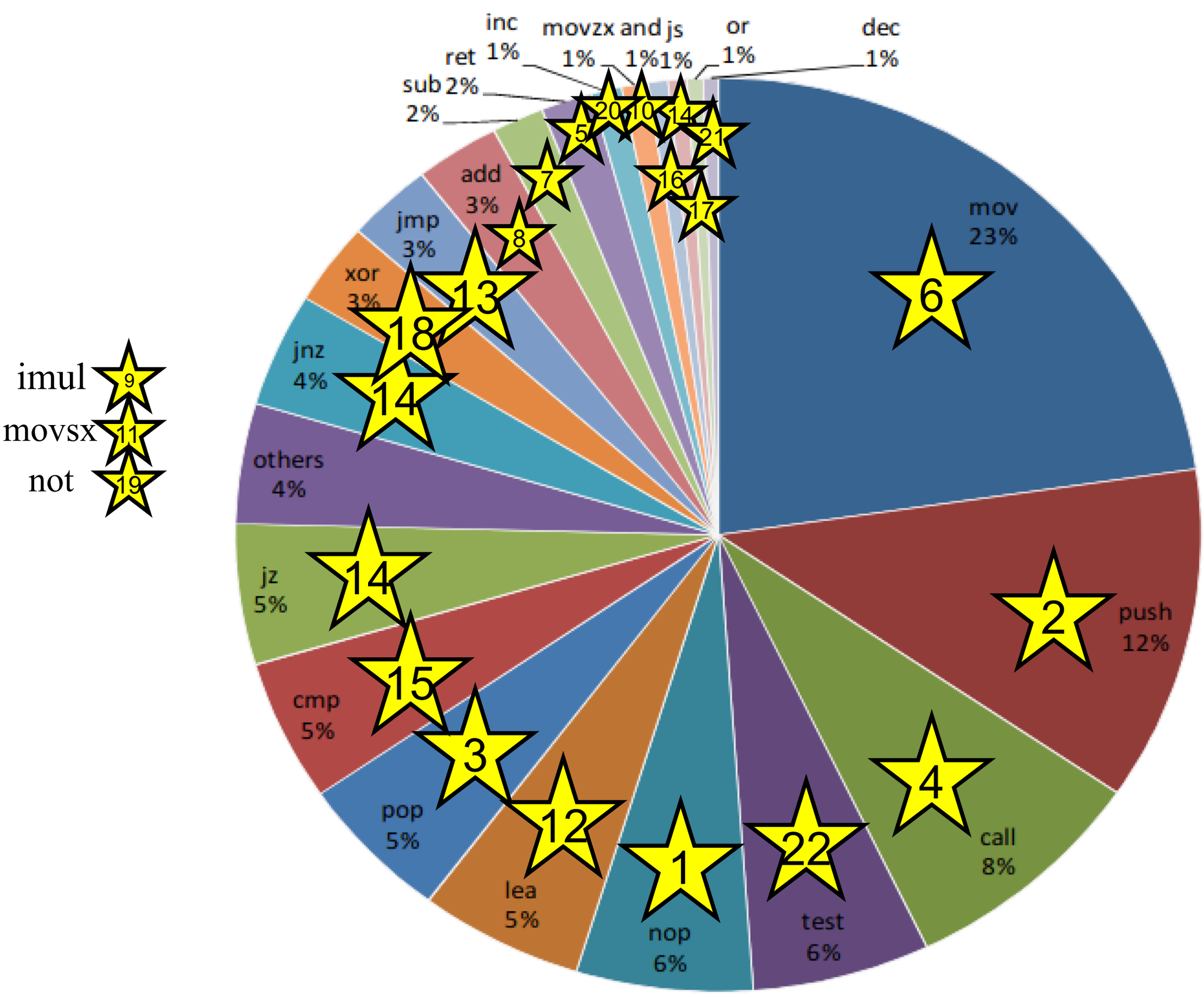


Fig. 2 Web browsers' instruction frequencies

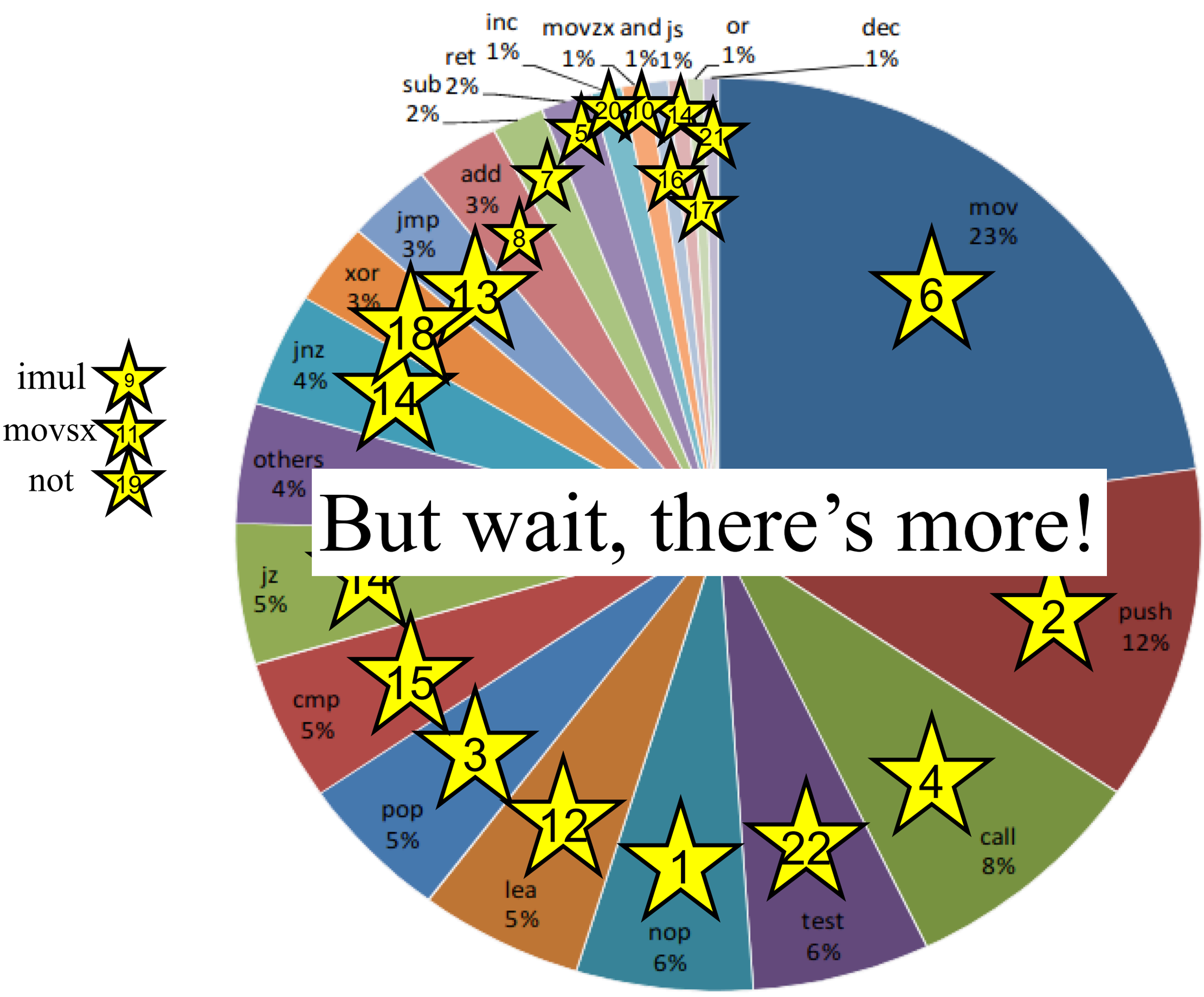


Fig. 2 Web browsers' instruction frequencies

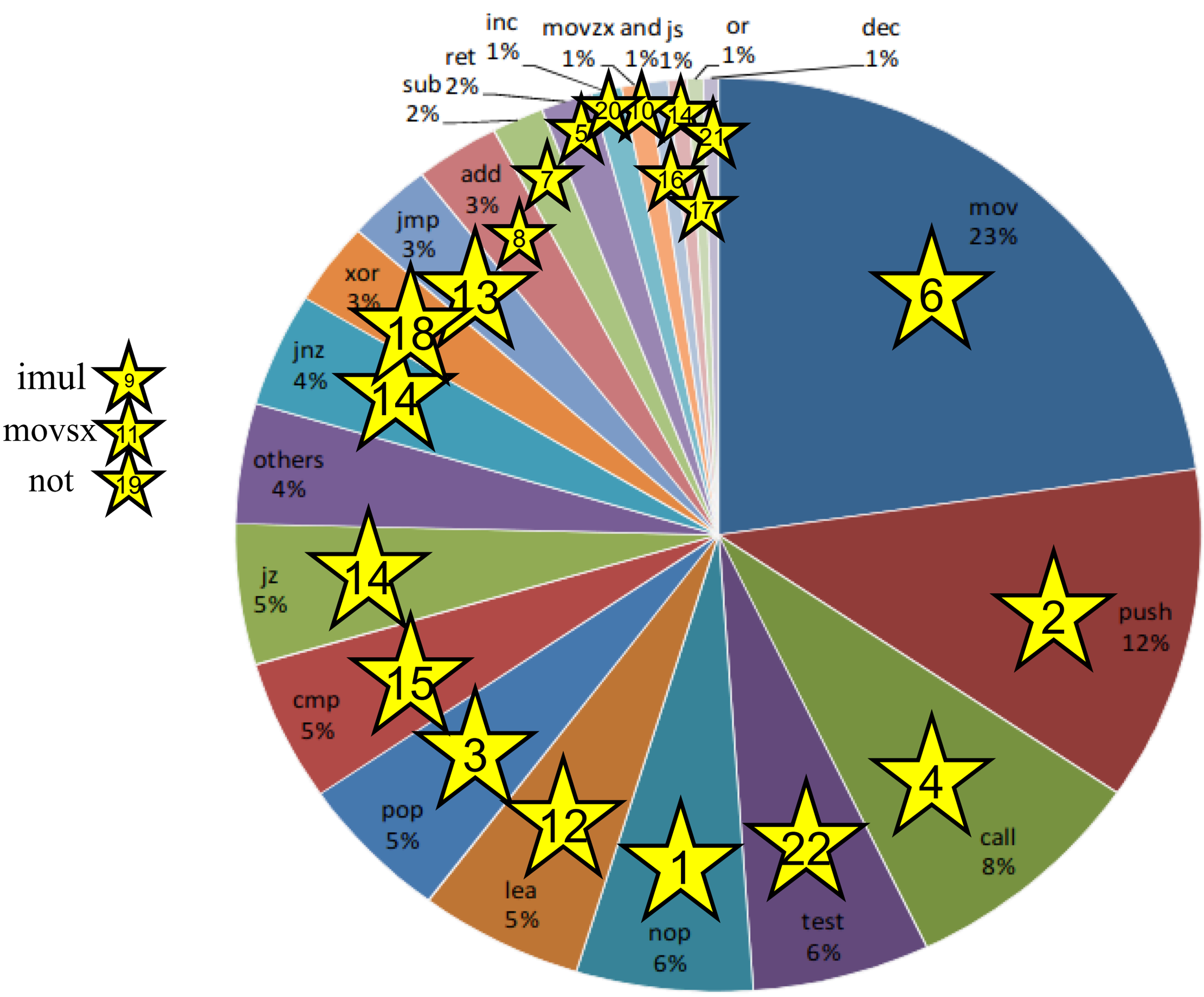


Fig. 2 Web browsers' instruction frequencies

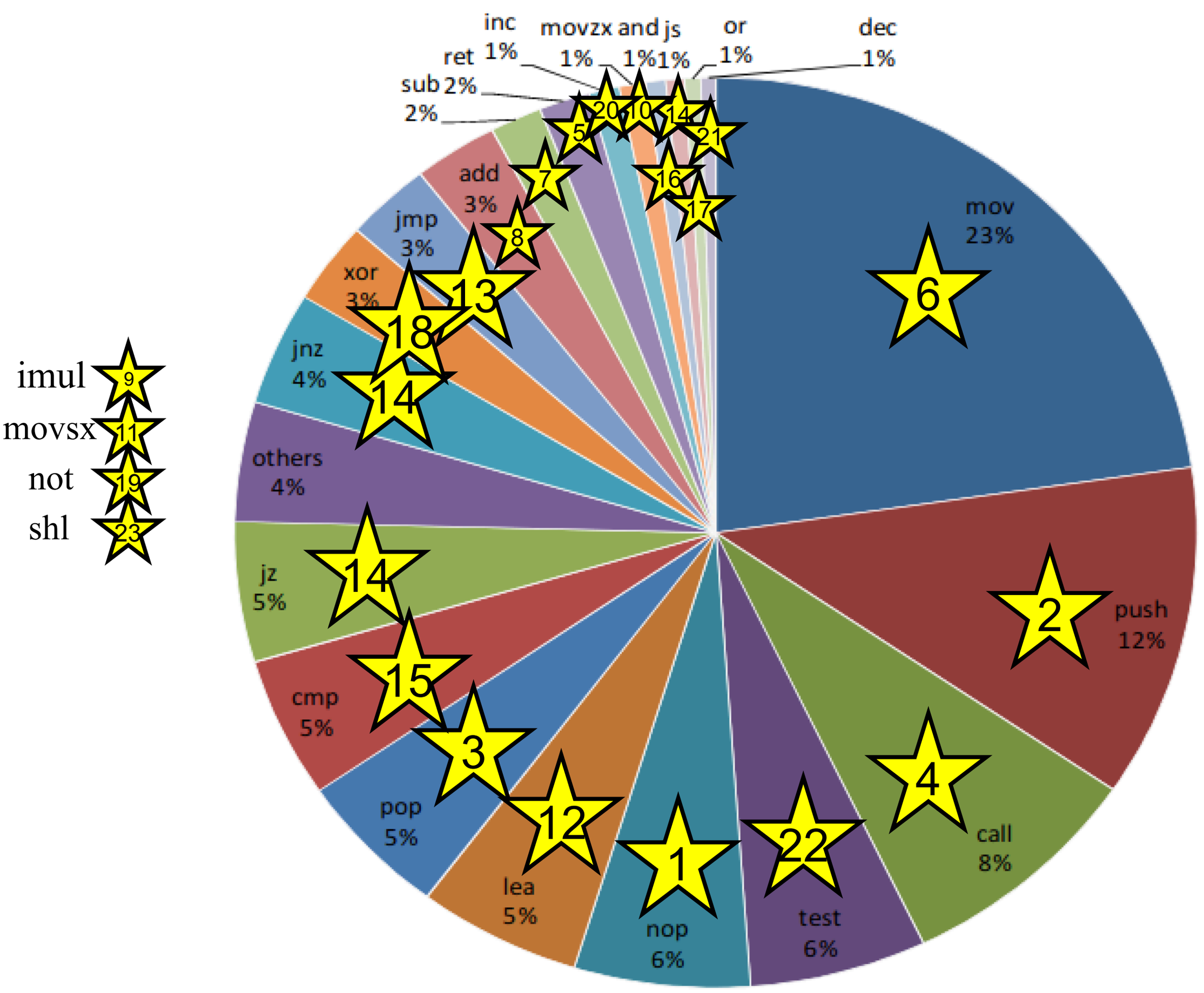


Fig. 2 Web browsers' instruction frequencies

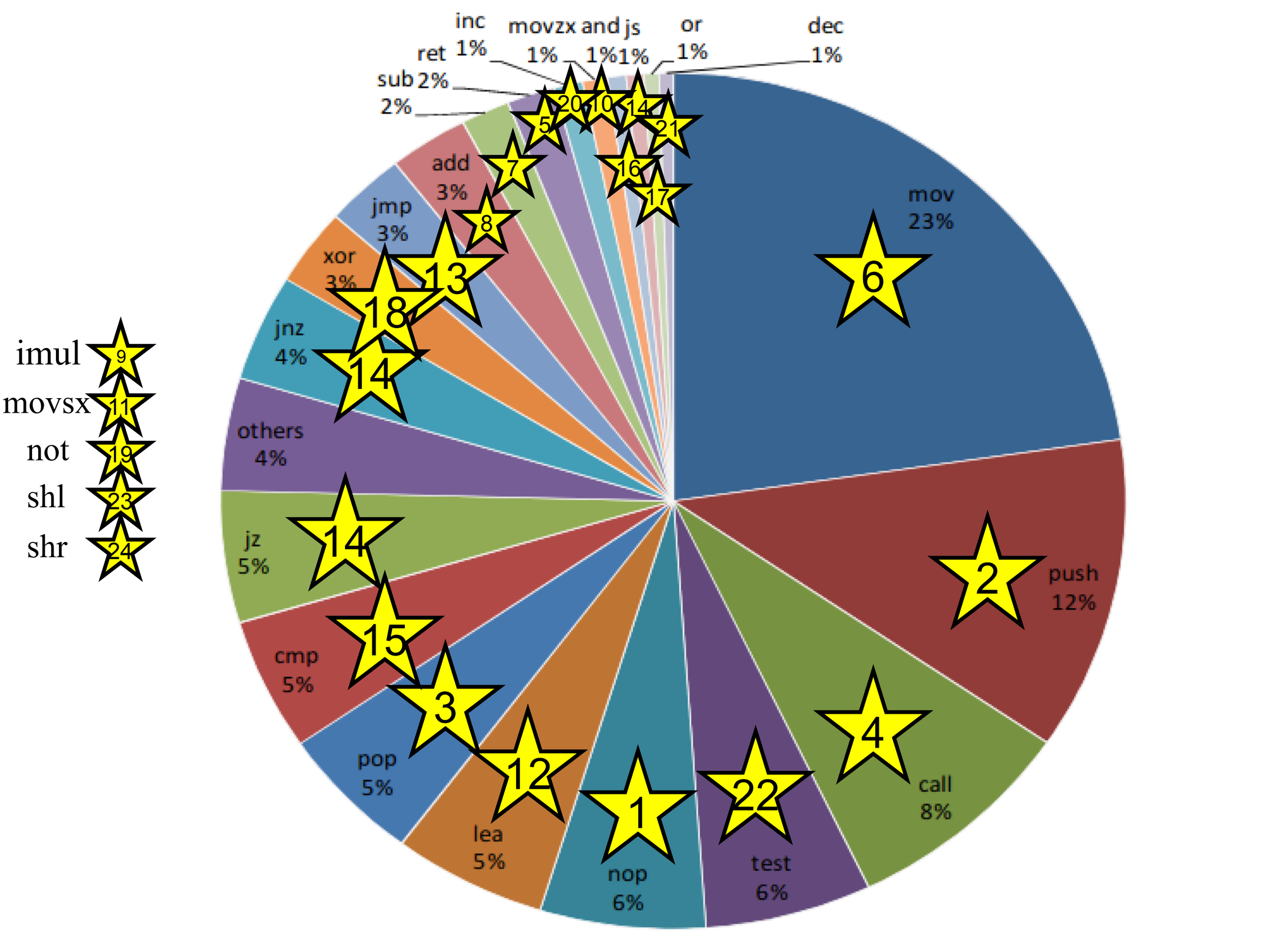


Fig. 2 Web browsers' instruction frequencies